Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

Отчёт по лабораторной работе №1 по дисциплине «Технологии и инструментальные средства проектирования интеллектуальных систем»

Предметная область «Языки программирования»

Выполнил: Казаченко В.А.
Гр. 321702

Проверил: Жмырко А.В.

Минск 2026

1 Цель работы

Изучить структуру и принципы работы баз знаний интеллектуальных систем, освоить навыки формализации предметных областей и фактографических высказываний в базах знаний ostis-систем, научиться навигироваться по базе знаний, выявлять закономерности и противоречия в ней.

2 Индивидуальный вариант

Вариант 26 — предметная область «Языки программирования».

Предметная область является слабо структурированной: существуют тысячи языков программирования, единой общепринятой классификации не сложилось, и один и тот же язык одновременно принадлежит нескольким группам. Так, Python является одновременно интерпретируемым языком, языком высокого уровня, языком с динамической типизацией и мультипарадигменным языком. Поэтому структуризация выполнена по нескольким независимым основаниям деления.

К объектам исследования отнесены сами языки программирования, а также сущности, без которых язык не может использоваться на практике: парадигмы программирования, реализации языка, стандарты, стандартные библиотеки, инструменты разработки, авторы языков и сопровождающие их организации. За пределами предметной области оставлены конкретные программы, написанные на этих языках, алгоритмы и аппаратное обеспечение.

3 Задание

— выбрать тему предметной области согласно варианту;
— формализовать структуру предметной области: выделить классы объектов исследования и исследуемые отношения;
— описать классы объектов исследования и исследуемые отношения — суммарно не менее 30 абсолютных и относительных понятий, задавая между ними теоретико-множественные отношения, а для отношений — свойства и домены; при описании понятий задавать примечания, пояснения и определения, пользуясь текстами из открытых источников;
— для каждого понятия описать не менее одного экземпляра и связать экземпляры между собой при помощи относительных понятий;
— протестировать полученные фрагменты базы знаний в системе NIKA;
— составить электронный отчёт;
— уметь отвечать на вопросы и решать задачи;

— запушить изменения в форк на GitHub.

4 Организация разработанного фрагмента базы знаний

Фрагмент базы знаний размещён в каталоге kb/extra проекта системы NIKA. Принято правило: один файл — одно понятие, имя файла совпадает с системным идентификатором внутри него. Структура каталогов приведена ниже.

```
kb/extra/section_subject_domain_of_programming_languages/
├── section_subject_domain_of_programming_languages.scs
├── concepts/                        23 абсолютных понятия
│   └── instances/                   40 экземпляров в 9 подпапках
│       ├── languages/               8 языков программирования
│       ├── paradigms/               4 парадигмы
│       ├── implementations/         3 реализации языков
│       ├── standards/               3 стандарта
│       ├── developers/              7 разработчиков
│       ├── organizations/           4 организации
│       ├── libraries/               2 стандартные библиотеки
│       ├── tools/                   4 инструмента разработки
│       └── years/                   5 календарных годов
└── relations/                       17 относительных понятий
```

Всего разработано 77 файлов с расширением .scs, содержащих 40 понятий — 23 абсолютных и 17 относительных — и 40 экземпляров, связанных между собой 70 связями.

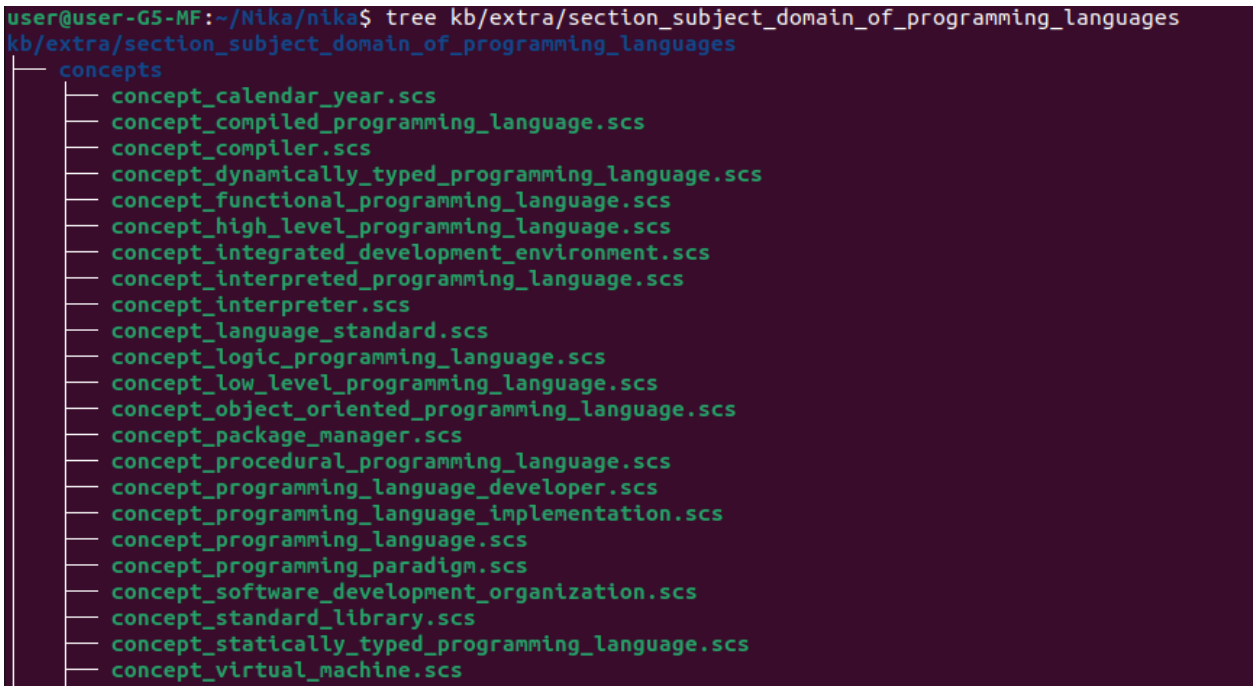


Рисунок 1 — Структура каталогов разработанного фрагмента базы знаний

5 Формализация структуры предметной области

Спецификация предметной области задана в файле раздела. Раздел объявлен атомарным, внутри его контура описан узел предметной области типа sc_node_struct, для которого указаны максимальные классы объектов исследования, немаксимальные классы и исследуемые отношения.

```
subject_domain_of_programming_languages
<- sc_node_struct;
<- subject_domain;
-> rrel_maximum_studied_object_class:
    concept_programming_language;
    concept_programming_paradigm;
    ...
-> rrel_not_maximum_studied_object_class:
    concept_compiled_programming_language;
    ...
-> rrel_explored_relation:
    nrel_language_author;
    ...;;
```

Таблица 1 — Максимальные классы объектов исследования

№	Понятие	Системный идентификатор
1	язык программирования	concept_programming_language
2	парадигма программирования	concept_programming_paradigm
3	реализация языка программирования	concept_programming_language_implementation
4	стандарт языка программирования	concept_language_standard
5	разработчик языка программирования	concept_programming_language_developer
6	организация-разработчик программного обеспечения	concept_software_development_organization
7	стандартная библиотека	concept_standard_library
8	интегрированная среда разработки	concept_integrated_development_environment
9	менеджер пакетов	concept_package_manager
10	календарный год	concept_calendar_year

Таблица 2 — Немаксимальные классы объектов исследования

№	Понятие	Системный идентификатор
1	компилируемый язык программирования	concept_compiled_programming_language
2	интерпретируемый язык программирования	concept_interpreted_programming_language
3	язык программирования высокого уровня	concept_high_level_programming_language
4	язык программирования низкого уровня	concept_low_level_programming_language
5	объектно-ориентированный язык	concept_object_oriented_programmi

№	Понятие	Системный идентификатор
	программирования	ng_language
6	функциональный язык программирования	concept_functional_programming_language
7	процедурный язык программирования	concept_procedural_programming_language
8	логический язык программирования	concept_logic_programming_language
9	язык программирования со статической типизацией	concept_statically_typed_programming_language
10	язык программирования с динамической типизацией	concept_dynamically_typed_programming_language
11	компилятор	concept_compiler
12	интерпретатор	concept_interpreter
13	виртуальная машина	concept_virtual_machine

Различие между максимальным и немаксимальным классом состоит в следующем: для максимального класса в данной предметной области отсутствует другой класс объектов исследования, являющийся его надмножеством. Так, понятие «язык программирования» является максимальным классом, а «компилируемый язык программирования» — немаксимальным, поскольку входит в него как подмножество.

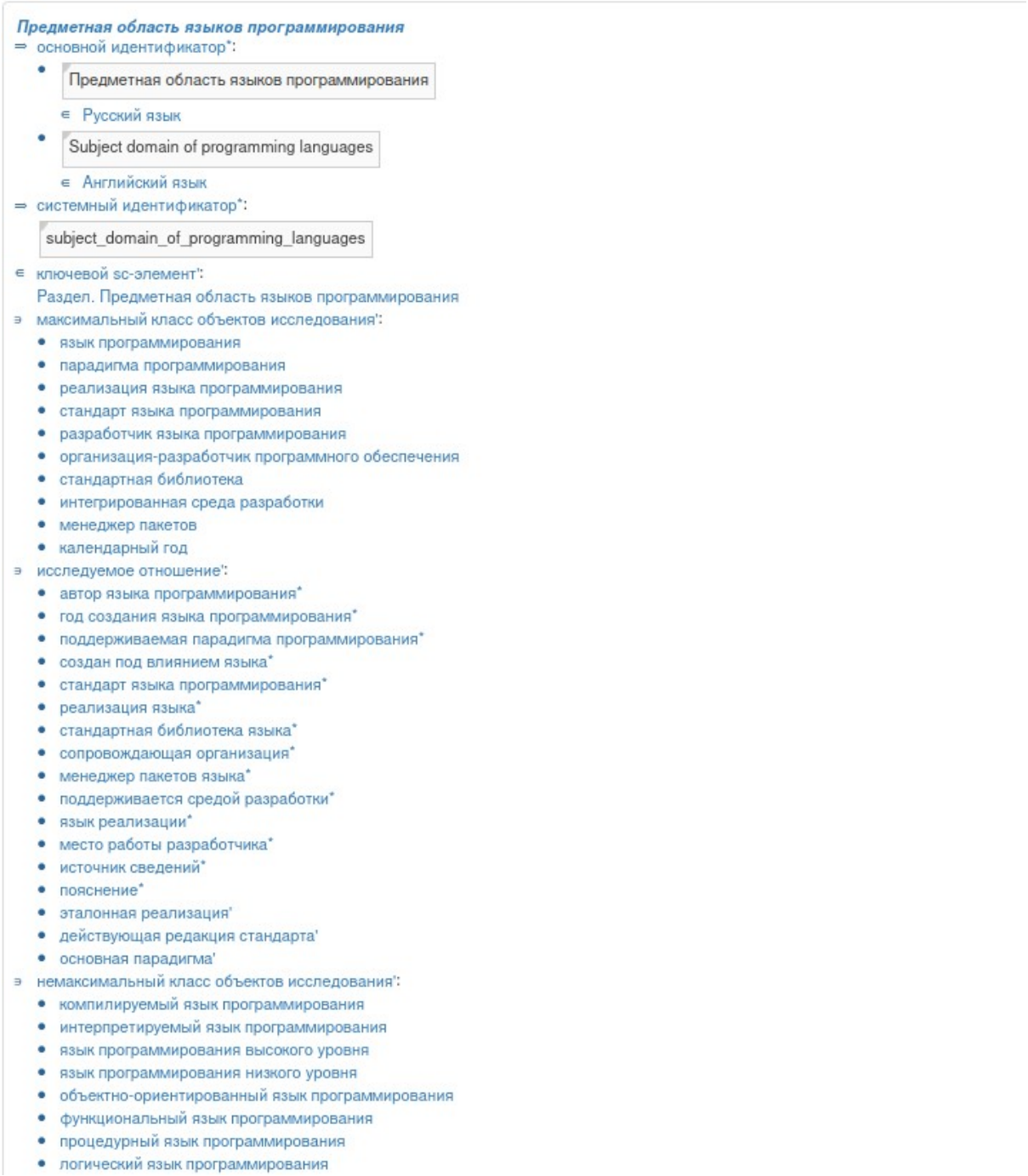


Рисунок 2 — Спецификация предметной области языков программирования

6 Описание абсолютных понятий

Каждое абсолютное понятие описано отдельным файлом и содержит: идентификаторы на русском и английском языках, примечание, пояснение, определение с переводом на естественный язык, ссылку на открытый источник и теоретико-множественные связи с другими понятиями. Ниже приведён сокращённый листинг понятия «компилятор».

```
concept_compiler
<- sc_node_class;
<- class;
<- concept_wit_entity;
=> nrel_main_idtf:
    [компилятор]
    (* <- lang_ru;; *);
    [compiler]
    (* <- lang_en;; *);
=> nrel_note:
    [Компилятор — программа, переводящая исходный текст программы с
языка
     программирования в эквивалентный код на языке более низкого
уровня.]
    (* <- lang_ru;; *);
=> nrel_explanation:
    [Компиляция выполняется один раз, поэтому готовая программа
работает
     быстрее, но её нужно пересобирать под каждую целевую
платформу.]
    (* <- lang_ru;; *);
<- rrel_key_sc_element:
    ...
    (*
        => nrel_main_idtf: [Опр. (компилятор)] (* <- lang_ru;; *);;
        <- definition;;
        <= nrel_sc_text_translation: ... ;;
    *);
=> nrel_source:
    [https://ru.wikipedia.org/wiki/Компилятор]
    (* <- lang_ru;; *);
<= nrel_inclusion:
    concept_programming_language_implementation;;
```

Для понятия «язык программирования» дополнительно заданы пять разбиений отношением nrel_subdividing — по способу выполнения программ, по уровню абстракции, по дисциплине типизации и по основной поддерживаемой парадигме. Каждое разбиение выполнено по одному основанию деления, что обеспечивает однородность выделяемых групп.

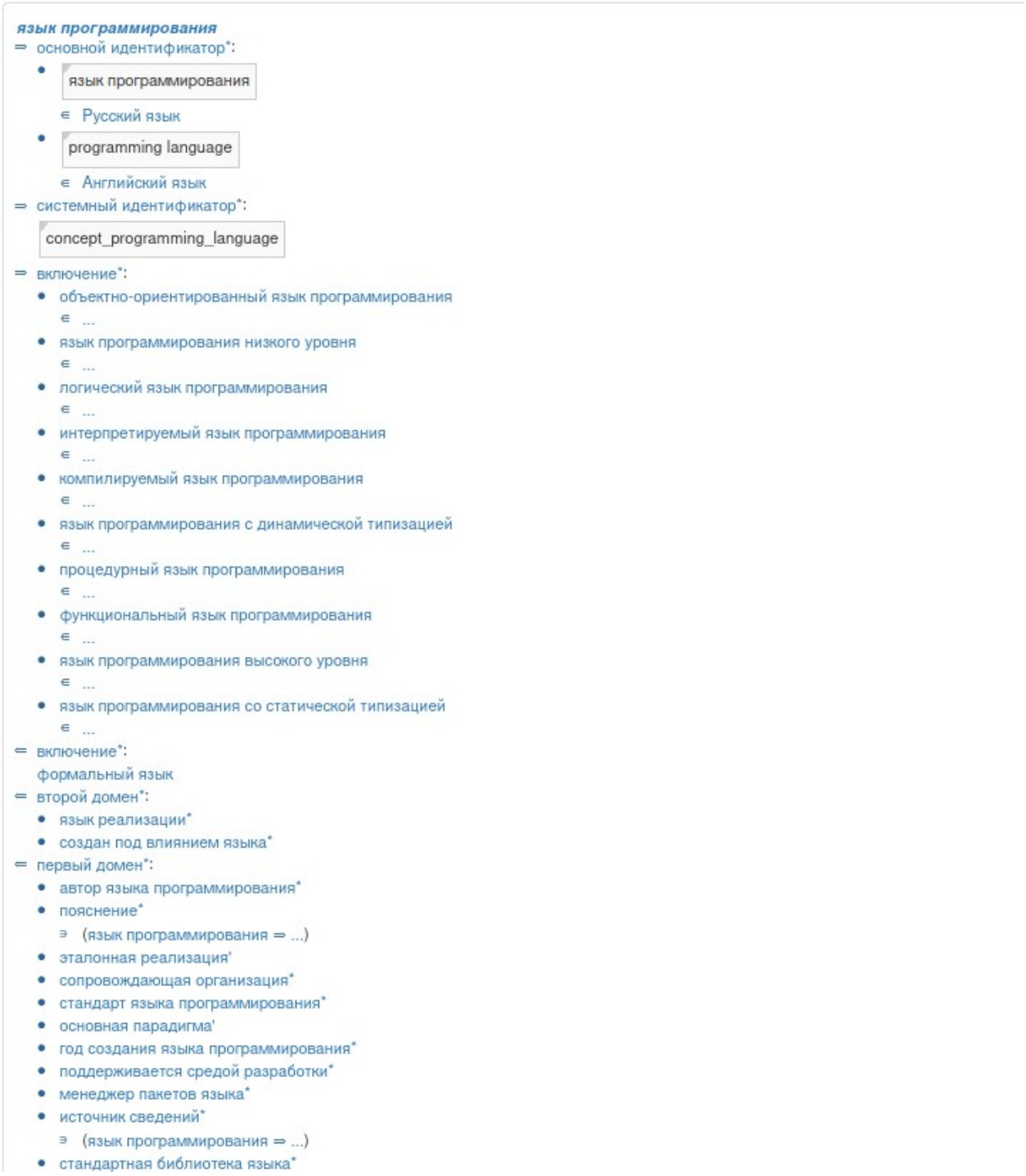


Рисунок 3 — Абсолютное понятие «язык программирования» и его разбиения

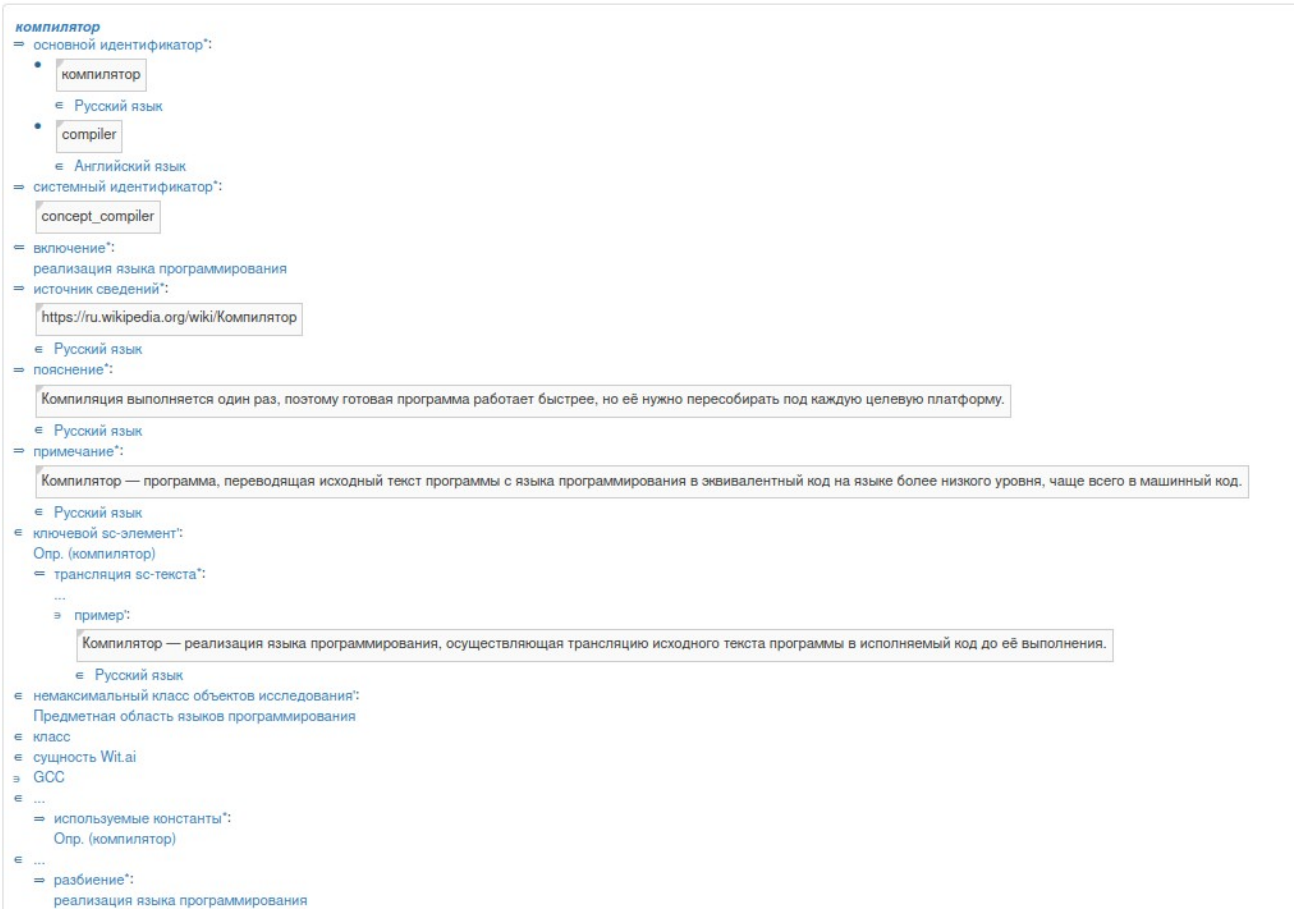


Рисунок 4 — Спецификация понятия: примечание, пояснение, определение и источник

7 Описание относительных понятий

Разработано 17 относительных понятий: 14 неролевых отношений и 3 ролевых. Для каждого неролевого отношения заданы свойства — бинарное, ориентированное, антирефлексивное, асимметричное, антитранзитивное — а также первый и второй домены и область определения через отношение nrel_combination.

Таблица 3 — Исследуемые отношения и их домены

Отношение	Первый домен	Второй домен
автор языка программирования*	язык программирования	разработчик языка
год создания языка программирования*	язык программирования	календарный год
поддерживаемая парадигма*	язык программирования	парадигма программирования
создан под влиянием языка*	язык программирования	язык программирования

Отношение	Первый домен	Второй домен
стандарт языка программирования*	язык программирования	стандарт языка
реализация языка*	язык программирования	реализация языка
стандартная библиотека языка*	язык программирования	стандартная библиотека
сопровождающая организация*	язык программирования	организация-разработчик
менеджер пакетов языка*	язык программирования	менеджер пакетов
поддерживается средой разработки*	язык программирования	среда разработки
язык реализации*	реализация языка	язык программирования
место работы разработчика*	разработчик языка	организация-разработчик
источник сведений*	язык программирования	файл (sc-ссылка)
пояснение*	язык программирования	файл (sc-ссылка)
эталонная реализация'	язык программирования	реализация языка
действующая редакция стандарта'	язык программирования	стандарт языка
основная парадигма'	язык программирования	парадигма программирования

Последние три отношения являются ролевыми: они представляют собой подмножество отношения принадлежности и указывают роль элемента внутри множества. В русском идентификаторе такие отношения оканчиваются апострофом, а системный идентификатор имеет префикс rrel_. Неролевые отношения оканчиваются звёздочкой и имеют префикс nrel_. Префиксы введены потому, что символы «*» и «'» не входят в разрешённый алфавит системных идентификаторов.

```
nrel_language_author
<- sc_node_norole_relation;
=> nrel_main_idtf:
    [автор языка программирования*]
    (* <- lang_ru;; *);
<- relation;
<- binary_relation;
<- oriented_relation;
<- antireflexive_relation;
<- asymmetric_relation;
<- antitransitive_relation;
=> nrel_first_domain: concept_programming_language;
=> nrel_second_domain: concept_programming_language_developer;
=> nrel_definitional_domain:
    ...
    (*
    <= nrel_combination:
        {
            concept_programming_language;
            concept_programming_language_developer
```

```
        };;
    *);;
```

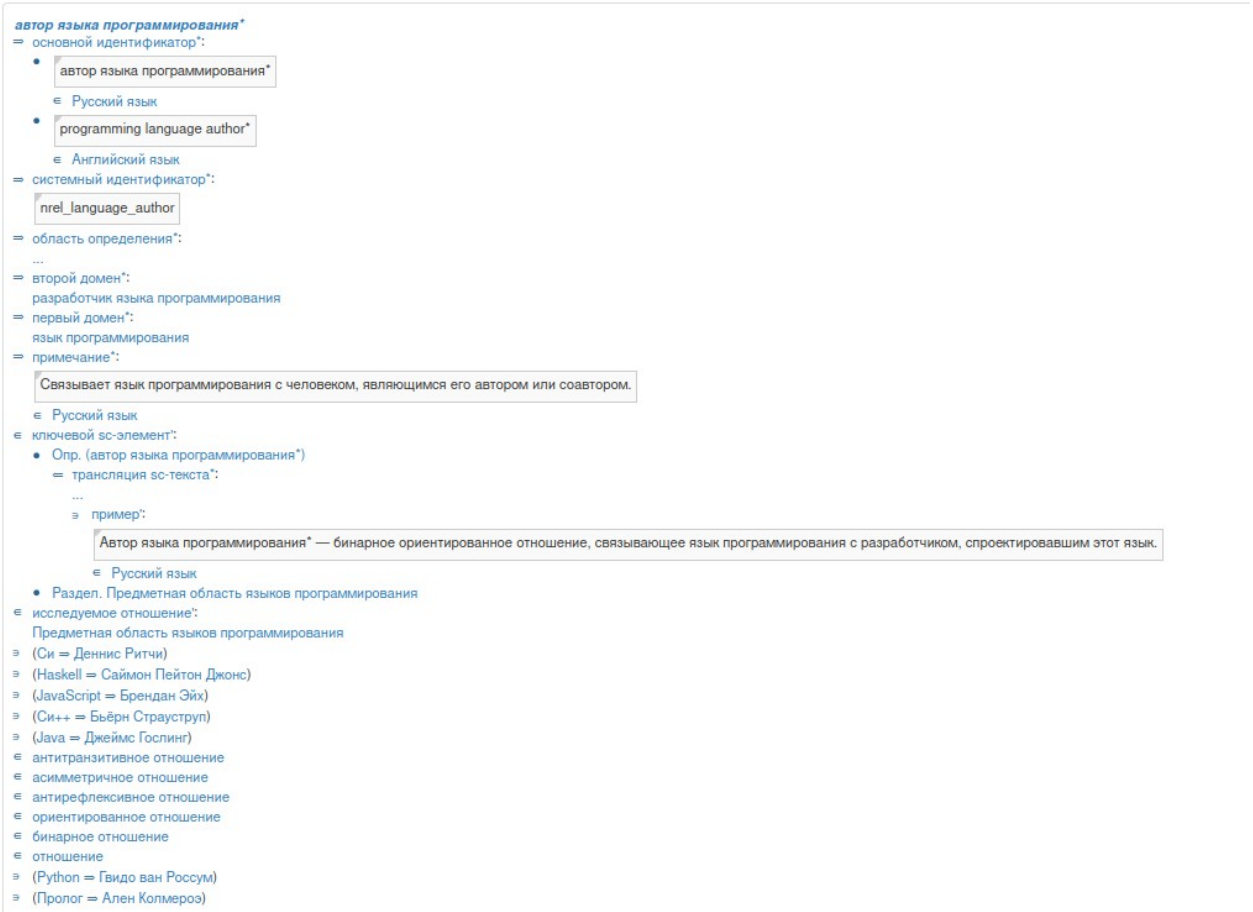


Рисунок 5 — Относительное понятие «автор языка программирования*»

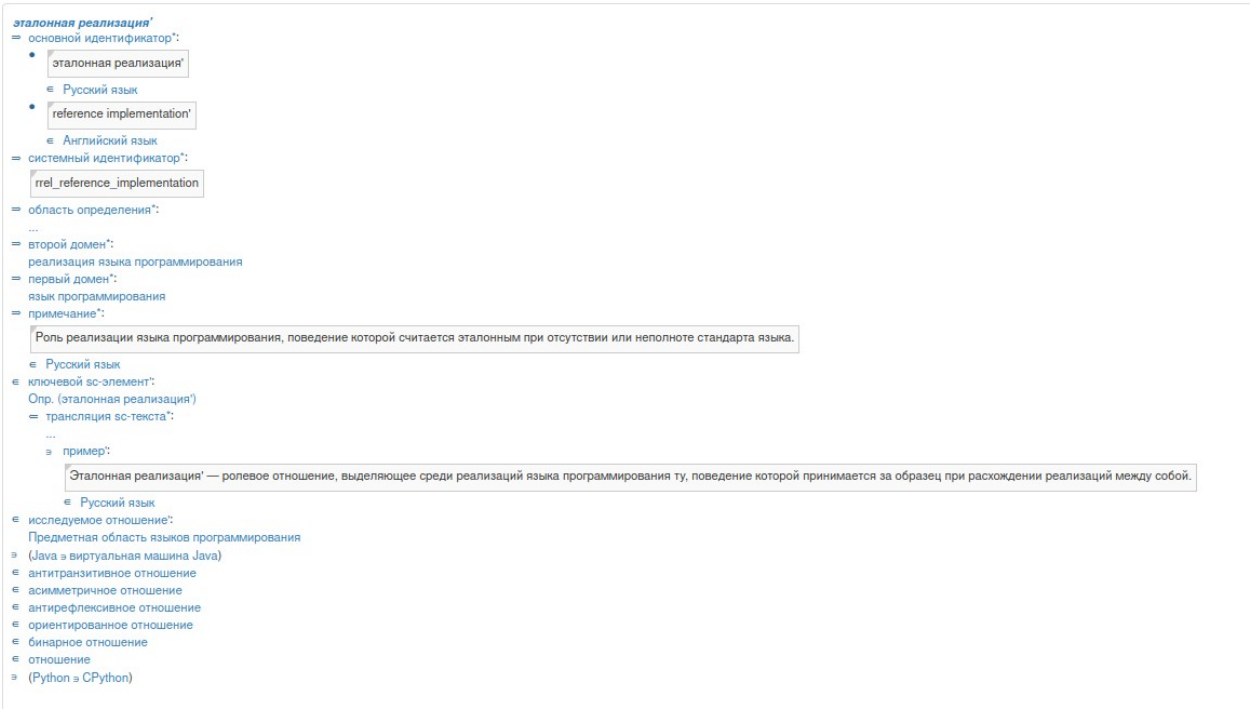


Рисунок 6 — Ролевое отношение «эталонная реализация'»

8 Описание экземпляров понятий

Для каждого понятия описано не менее одного экземпляра — всего 40. Экземпляры связаны между собой относительными понятиями, что образует связный граф: изолированных узлов нет, каждое отношение задействовано не менее двух раз. Ниже приведён листинг экземпляра «Python».

```
python_language
<- concept_programming_language;
<- concept_interpreted_programming_language;
<- concept_high_level_programming_language;
<- concept_dynamically_typed_programming_language;
<- concept_object_oriented_programming_language;
=> nrel_main_idtf: [Python] (* <- lang_ru;; *);
=> nrel_idtf: [Питон] (* <- lang_ru;; *);
=> nrel_language_author: guido_van_rossum;
=> nrel_creation_year: year_1991;
=> nrel_supported_paradigm: paradigm_object_oriented_programming;
    paradigm_functional_programming;
    paradigm_procedural_programming;
=> nrel_influenced_by: c_language;
=> nrel_language_implementation: cpython_interpreter;
=> nrel_maintaining_organization: python_software_foundation;
-> rrel_reference_implementation: cpython_interpreter;
-> rrel_main_paradigm: paradigm_object_oriented_programming;;
```

Связность экземпляров иллюстрируется цепочками переходов. Например: Python — автор — Гвидо ван Россум — место работы — Python Software Foundation. Или: Python — создан под влиянием — Си — стандарт

языка — ISO/IEC 9899:2018. Наиболее связанными узлами являются Си и Python — по 14 связей каждый.

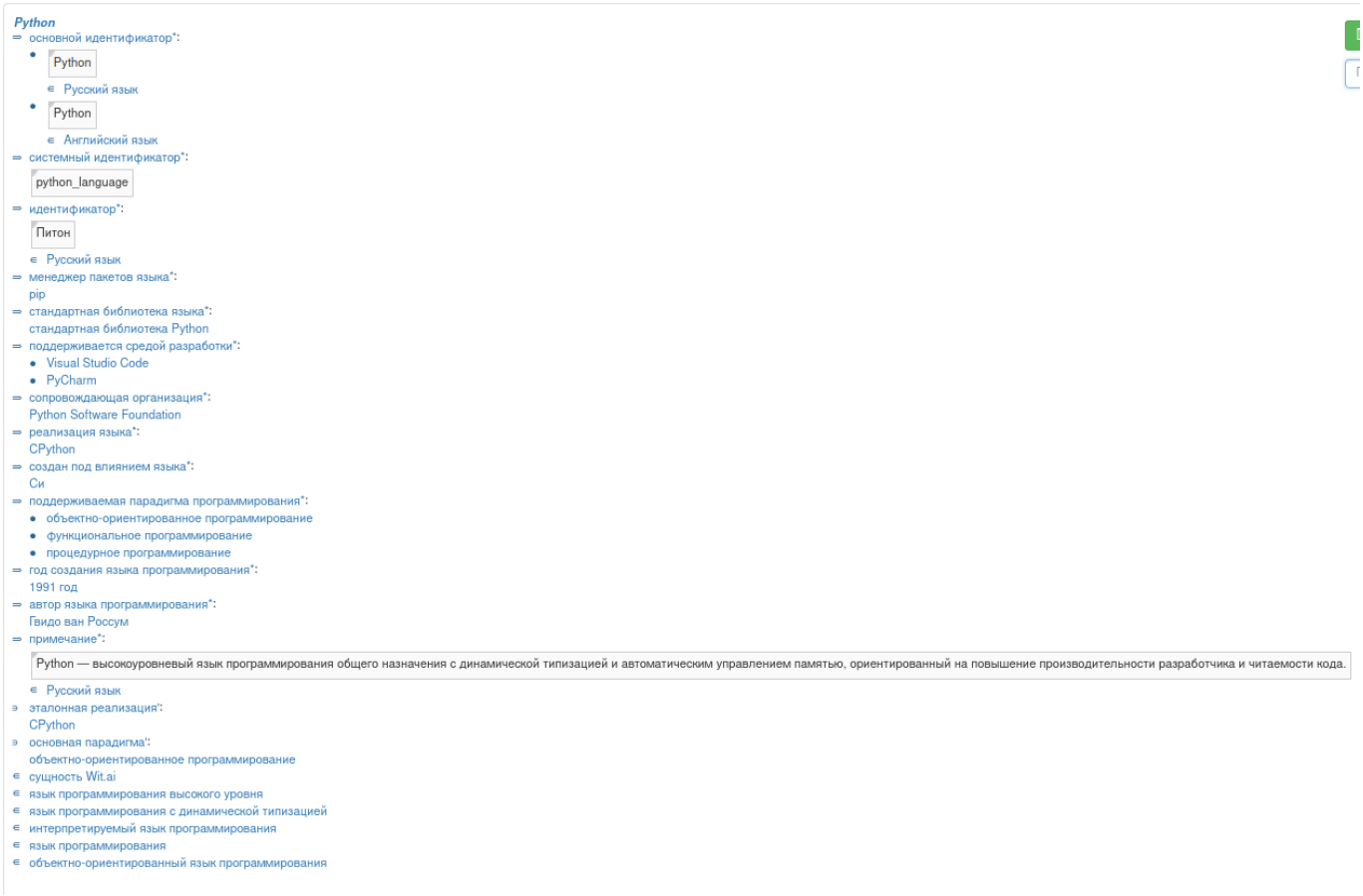


Рисунок 7 — Семантическая окрестность экземпляра «Python»

9 Тестирование фрагментов базы знаний

База знаний пересобирается из исходных текстов при каждом запуске системы. Успешность сборки проверяется по журналу решателя: в нём должны присутствовать упоминания файлов разработанного фрагмента и отсутствовать строки с ошибками.

```
docker compose logs problem-solver | grep -i \
    'section_subject_domain_of_programming_languages'

docker compose logs problem-solver | grep -i error
```

Навигация по базе знаний выполнялась средствами веб-интерфейса sc-web: поиск понятий по системному идентификатору, просмотр семантической окрестности, переключение между представлениями SCn и SCg, переходы по связям через контекстное меню. Все разработанные понятия и экземпляры доступны для поиска и корректно отображаются.

Следует отметить, что распознавание вопросов на естественном языке обеспечивается классификатором сообщений, обучение которого выполняется в рамках лабораторной работы №2. До её выполнения система может не распознавать вопросы по разработанной предметной области, при этом сами знания в базе знаний присутствуют и доступны для просмотра через интерфейс sc-web.

10 Выводы

В ходе выполнения лабораторной работы изучены структура и принципы организации баз знаний ostis-систем, освоена формализация предметной области на языке SCs и навигация по базе знаний средствами интерфейса sc-web.

Разработан фрагмент базы знаний системы NIKA по предметной области «Языки программирования», содержащий 40 понятий — 23 абсолютных и 17 относительных — и 40 экземпляров, связанных 70 связями. Требование методических указаний о наличии не менее 30 понятий перевыполнено.

Основной вывод содержательного характера состоит в том, что выбранная предметная область не допускает структуризации по единственному основанию: языки программирования обладают несколькими независимыми характеристиками, поэтому один и тот же язык одновременно принадлежит нескольким классам. Корректным решением является выделение нескольких независимых оснований деления, каждое из которых задаёт собственное однородное разбиение. В разработанном фрагменте это отражено пятью разбиениями отношением nrel_subdividing.

Практическую сложность представляло соблюдение направления дуг: отношение включения задаётся связкой nrel_inclusion, и перестановка знаков «=>» и «<=» приводит к тому, что надкласс и подкласс меняются местами. При этом база знаний собирается без ошибок, а искажение иерархии обнаруживается только при просмотре семантической окрестности.

Полученный фрагмент базы знаний является основой для последующих лабораторных работ: описанные понятия и экземпляры будут использоваться при обучении классификатора сообщений и при проектировании логических правил построения диалога.

11. Список использованных источников

Стандарт Технологии OSTIS. URL: https://libeldoc.bsuir.by/bitstream/123456789/45813/1/Golenkov_Otkrytaya.pdf
Метасистема OSTIS. URL: http://ims.ostis.net
Документация системы NIKA. URL: https://ostis-apps.github.io/nika/
Репозиторий системы NIKA. URL: https://github.com/ostis-apps/nika
Языки программирования — Википедия. URL: https://ru.wikipedia.org/wiki/Язык_программирования